In [12]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense
```

In [15]:
```
data = pd.read_csv("cifar-10.csv")
```

In [16]:
```
X = data.drop("label", axis=1).values
y = data["label"].values
```

In [17]:
```
X = X.reshape(-1, 32, 32, 3)
```

In [18]:
```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.1, random_state=42, stratify=y
)
```

In [19]:
```
X_train = X_train.astype("float32") / 255.0
X_test = X_test.astype("float32") / 255.0

print("Training data:", X_train.shape)
print("Testing data:", X_test.shape)
```

```
Training data: (45000, 32, 32, 3)
Testing data: (5000, 32, 32, 3)
```

In [21]:
```
model = Sequential([
    Conv2D(32, (3, 3), activation="relu", input_shape=(32, 32, 3)),
    MaxPooling2D((2, 2)),

    Conv2D(64, (3, 3), activation="relu"),
    MaxPooling2D((2, 2)),

    Flatten(),
    Dense(128, activation="relu"),
```

```
    Dense(10, activation="softmax")
])

model.summary()
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
conv2d_2 (Conv2D)	(None, 30, 30, 32)	896
max_pooling2d_2 (MaxPooling2D)	(None, 15, 15, 32)	0
conv2d_3 (Conv2D)	(None, 13, 13, 64)	18,496
max_pooling2d_3 (MaxPooling2D)	(None, 6, 6, 64)	0
flatten_1 (Flatten)	(None, 2304)	0
dense_2 (Dense)	(None, 128)	295,040
dense_3 (Dense)	(None, 10)	1,290

Total params: 315,722 (1.20 MB)

Trainable params: 315,722 (1.20 MB)

Non-trainable params: 0 (0.00 B)

In [23]:
```
model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)
```

In [24]:
```
history = model.fit(
    X_train,
    y_train,
    epochs=10,
    batch_size=64,
```

```
    validation_split=0.1
)
```

```
Epoch 1/10
633/633 ━━━━━━━━━━━━━━━━━━━━ 9s 12ms/step - accuracy: 0.4272 - loss: 1.6090 - val_accuracy: 0.5127 - val_loss: 1.3972
Epoch 2/10
633/633 ━━━━━━━━━━━━━━━━━━━━ 7s 12ms/step - accuracy: 0.5421 - loss: 1.2996 - val_accuracy: 0.5580 - val_loss: 1.2612
Epoch 3/10
633/633 ━━━━━━━━━━━━━━━━━━━━ 8s 13ms/step - accuracy: 0.5866 - loss: 1.1800 - val_accuracy: 0.5804 - val_loss: 1.2161
Epoch 4/10
633/633 ━━━━━━━━━━━━━━━━━━━━ 10s 16ms/step - accuracy: 0.6205 - loss: 1.0856 - val_accuracy: 0.6016 - val_loss: 1.1551
Epoch 5/10
633/633 ━━━━━━━━━━━━━━━━━━━━ 10s 16ms/step - accuracy: 0.6499 - loss: 1.0017 - val_accuracy: 0.6122 - val_loss: 1.1016
Epoch 6/10
633/633 ━━━━━━━━━━━━━━━━━━━━ 10s 16ms/step - accuracy: 0.6710 - loss: 0.9397 - val_accuracy: 0.6222 - val_loss: 1.0935
Epoch 7/10
633/633 ━━━━━━━━━━━━━━━━━━━━ 10s 15ms/step - accuracy: 0.6920 - loss: 0.8820 - val_accuracy: 0.6238 - val_loss: 1.0874
Epoch 8/10
633/633 ━━━━━━━━━━━━━━━━━━━━ 9s 15ms/step - accuracy: 0.7114 - loss: 0.8287 - val_accuracy: 0.6331 - val_loss: 1.0831
Epoch 9/10
633/633 ━━━━━━━━━━━━━━━━━━━━ 10s 16ms/step - accuracy: 0.7309 - loss: 0.7739 - val_accuracy: 0.6260 - val_loss: 1.1204
Epoch 10/10
633/633 ━━━━━━━━━━━━━━━━━━━━ 11s 17ms/step - accuracy: 0.7458 - loss: 0.7274 - val_accuracy: 0.6176 - val_loss: 1.1779
```

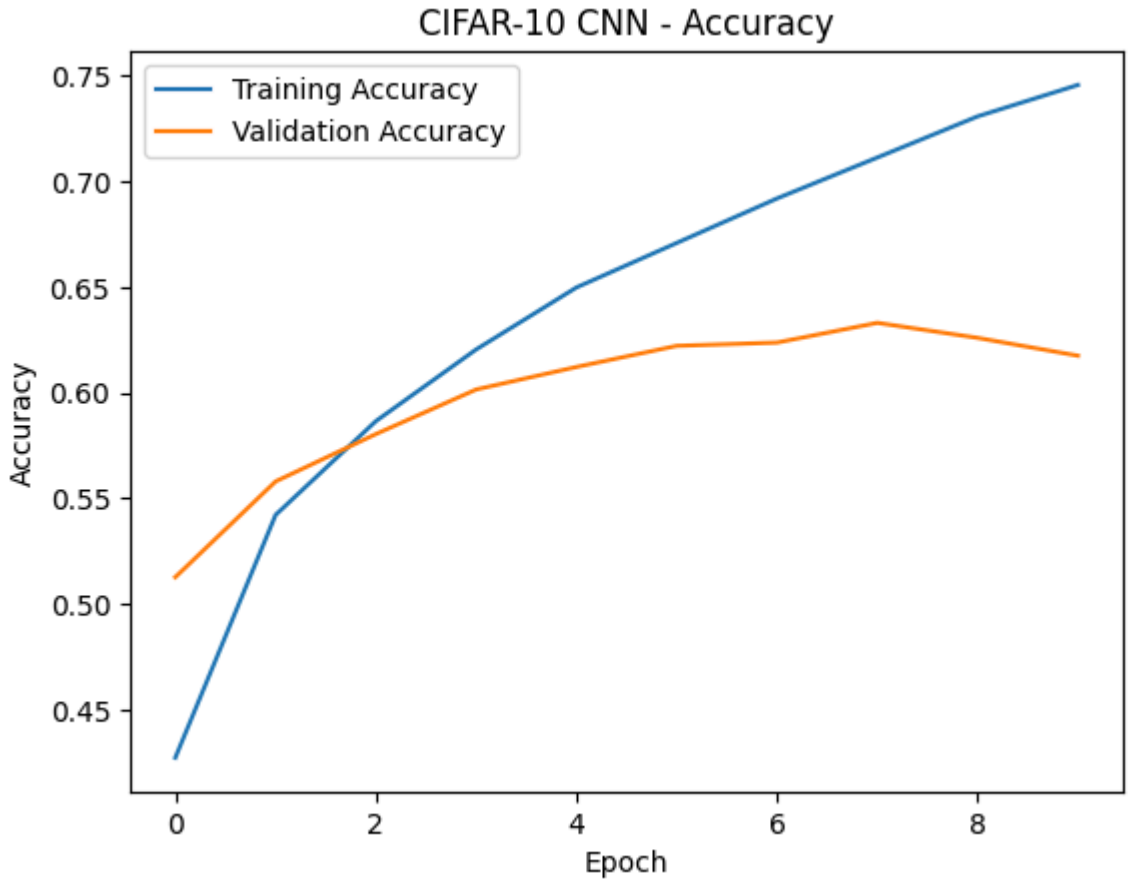
In [25]:
```
test_loss, test_accuracy = model.evaluate(X_test, y_test, verbose=0)

print("Test Loss:", test_loss)
print("Test Accuracy:", test_accuracy * 100, "%")
```

```
Test Loss: 1.1594990491867065
Test Accuracy: 62.09999918937683 %
```

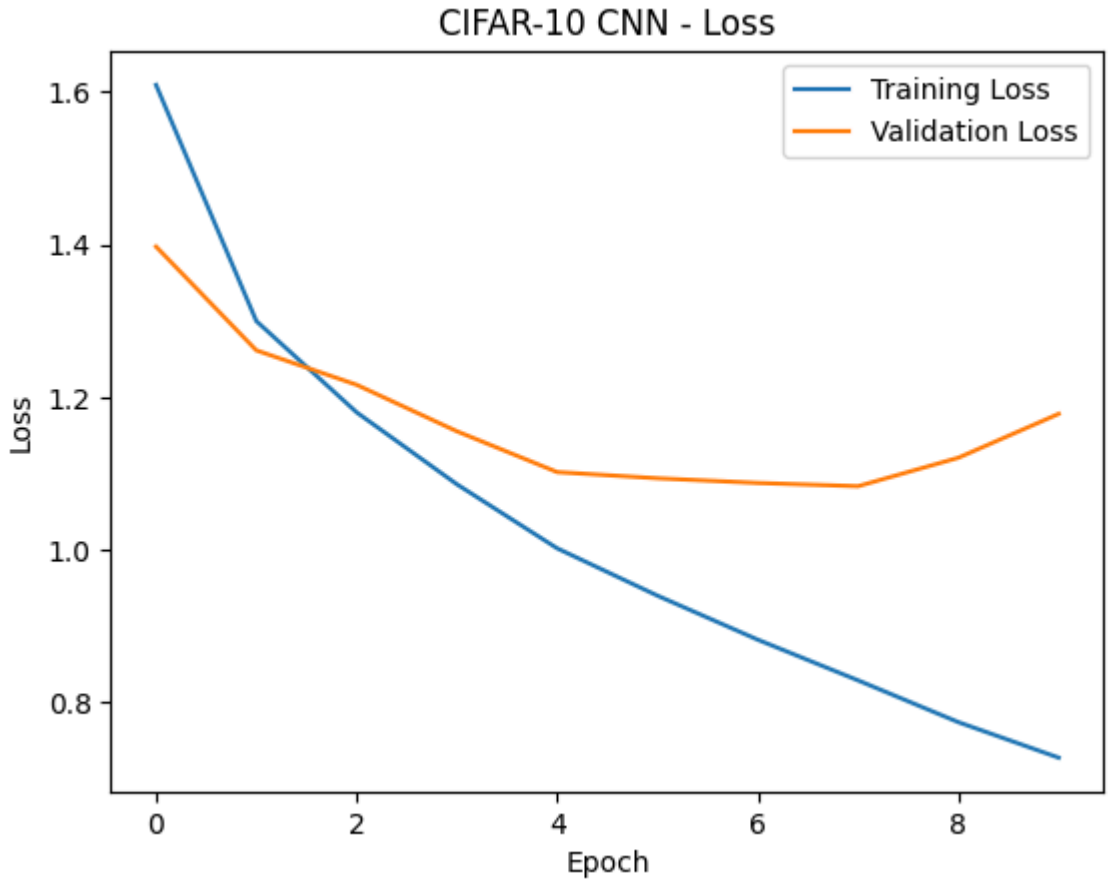
In [26]:
```
plt.plot(history.history["accuracy"], label="Training Accuracy")
plt.plot(history.history["val_accuracy"], label="Validation Accuracy")

plt.title("CIFAR-10 CNN - Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.show()
```



In [27]:
```
plt.plot(history.history["loss"], label="Training Loss")
plt.plot(history.history["val_loss"], label="Validation Loss")

plt.title("CIFAR-10 CNN - Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.show()
```


CIFAR-10 CNN - Loss
Training Loss
Validation Loss
Loss
Epoch
1.6
1.4
1.2
1.0
0.8
0
2
4
6
8

In [ ]: